SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I Kanish Kumar V (192524361) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kanish Kumar V

RUBRICS FOR CALCULATION:

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Compari son Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidenc e	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presenta tion	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Sequential File Organization
Records are stored in a specific order, usually based on a key field. It provides efficient sequential and ordered retrieval, but inserting and deleting records can be expensive because the order must be maintained.

Heap File Organization
Records are stored without any particular order. Insertion is very fast because new records can be added at the end or in available space. However, searching and deletion are slower because the system may need to scan many records.

Performance Comparison

Operation	Sequential File	Heap File
Insertion	Slow	**Fast**
Deletion	Slow	Moderate
Sequential Retrieval	**Fast**	Moderate
Specific Record Search	Fast if ordered	Slow
Maintaining Order	Required	Not required

Conclusion
Sequential files are better when records are frequently retrieved in sorted or sequential order. Heap files are better when fast insertion is the main requirement and records do not need to be maintained in a particular order.

```
mysql> CREATE TABLE students (
    ->     student_id INT PRIMARY KEY,
    ->     student_name VARCHAR(100),
    ->     department VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> -- Insert records
mysql> INSERT INTO students VALUES
    -> (103, 'Arun', 'CSE'),
    -> (101, 'Kavin', 'ECE'),
    -> (102, 'Divya', 'IT');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Sequential-style access
mysql> SELECT *
    -> FROM students
    -> ORDER BY student_id;
+------------+--------------+------------+
| student_id | student_name | department |
+------------+--------------+------------+
|        101 | Kavin        | ECE        |
|        102 | Divya        | IT         |
|        103 | Arun         | CSE        |
+------------+--------------+------------+
3 rows in set (0.00 sec)

mysql>
mysql> -- Heap-style retrieval without ordering
mysql> SELECT *
    -> FROM students;
+------------+--------------+------------+
| student_id | student_name | department |
+------------+--------------+------------+
|        101 | Kavin        | ECE        |
|        102 | Divya        | IT         |
|        103 | Arun         | CSE        |
+------------+--------------+------------+
3 rows in set (0.00 sec)

mysql>
mysql> -- Insert a new record
mysql> INSERT INTO students VALUES
    -> (104, 'Rahul', 'CSE');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- Delete a record
mysql> DELETE FROM students
    -> WHERE student_id = 102;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- Retrieve a specific record
mysql> SELECT *
    -> FROM students
```

2. Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

Clustered Indexing

A clustered index determines the physical order of records in a table. A table can have only one clustered index because the data can be physically ordered in only one way.

```
mysql> CREATE TABLE students (
    ->     student_id INT PRIMARY KEY,
    ->     student_name VARCHAR(100),
    ->     department VARCHAR(50),
    ->     cgpa DECIMAL(3,2)
    -> );
ERROR 1050 (42S01): Table 'students' already exists
mysql>
mysql> CREATE INDEX idx_student_id
    -> ON students(student_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_department
    -> ON students(department);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE INDEX idx_cgpa
    -> ON students(cgpa);
ERROR 1072 (42000): Key column 'cgpa' doesn't exist in table
mysql> SELECT *
    -> FROM students
    -> WHERE department = 'CSE';
+------------+--------------+------------+
| student_id | student_name | department |
+------------+--------------+------------+
|        103 | Arun         | CSE        |
|        104 | Rahul        | CSE        |
+------------+--------------+------------+
2 rows in set (0.01 sec)

mysql>
mysql> SELECT *
    -> FROM students
    -> WHERE cgpa BETWEEN 8.0 AND 9.0;
```

Difference

Feature	Clustered Index	Non-Clustered Index
Data order	Physically ordered	Separate index structure
Number per table	Usually one	Multiple
Storage	Data and index closely associated	Additional storage required
Range queries	Very efficient	Efficient
Example	`student_id`	`department`, `cgpa`

Conclusion

Clustered indexing is suitable for the primary ordering of data, while **non-clustered indexing** is useful for additional search fields. For example, `student_id` can be clustered, while `department` and `cgpa` can have non-clustered indexes.

3. Compare primary indexing and secondary indexing based on storage requirements and search performance.

Primary Indexing

Primary indexing is created on the **primary key or ordering key** of a sorted file. It usually requires less storage because it can be a **sparse index**, containing one index entry for each data block.

```
mysql> CREATE TABLE students (
    ->     student_id INT PRIMARY KEY,
    ->     student_name VARCHAR(100),
    ->     department VARCHAR(50),
    ->     cgpa DECIMAL(3,2)
    -> );
ERROR 1050 (42S01): Table 'students' already exists
mysql>
mysql> CREATE INDEX idx_student_id
    -> ON students(student_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_department
    -> ON students(department);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE INDEX idx_cgpa
    -> ON students(cgpa);
ERROR 1072 (42000): Key column 'cgpa' doesn't exist in table
mysql> SELECT *
    -> FROM students
    -> WHERE department = 'CSE';
+------------+--------------+------------+
| student_id | student_name | department |
+------------+--------------+------------+
|        103 | Arun         | CSE        |
|        104 | Rahul        | CSE        |
+------------+--------------+------------+
2 rows in set (0.01 sec)
```

Comparison

Feature	Primary Index	Secondary Index
Created on	Primary/ordering key	Non-ordering field
Storage	**Less**	**More**
Index type	Usually sparse	Usually dense
Search performance	Very fast	Very fast
Number of indexes	Usually one	Multiple possible

Conclusion

Primary indexing requires less storage because it can use a sparse index, while **secondary indexing requires more storage** but provides fast access through additional attributes. Both improve search performance compared with a full file scan.

4. Analyse the differences between static hashing and dynamic hashing in database systems.

Static Hashing

In static hashing, the number of buckets is fixed. A hash function maps each record to a fixed bucket. When the number of records increases, bucket overflow may occur.

```
mysql> CREATE TABLE employees (
    ->     employee_id INT PRIMARY KEY,
    ->     employee_name VARCHAR(100),
    ->     department VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE INDEX idx_employee
    -> ON employees(employee_id);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT *
    -> FROM employees
    -> WHERE employee_id = 105;
```

Comparison

Feature	Static Hashing	Dynamic Hashing
Number of buckets	Fixed	Dynamic
Overflow	Overflow buckets	Bucket splitting
Performance	Decreases with overflow	More consistent
Storage	Lower	Higher due to directory
Implementation	Simple	More complex
Best suited for	Stable data	Frequently growing data

Conclusion

Static hashing is suitable when the number of records is relatively stable. Dynamic hashing is better for databases where records are frequently inserted or deleted because it can adjust the number of buckets dynamically and reduce overflow.

5. Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

B+ Tree Indexing

A B+ Tree stores data pointers only in the leaf nodes. The leaf nodes are linked, making sequential and range searches efficient.

B-Tree Indexing

A **B-Tree** stores both keys and data pointers in internal nodes as well as leaf nodes. Searching can finish at any level of the tree.

```
mysql> CREATE TABLE products (
    ->     product_id INT PRIMARY KEY,
    ->     product_name VARCHAR(100),
    ->     price DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE INDEX idx_price
    -> ON products(price);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT *
    -> FROM products
    -> WHERE price BETWEEN 5000 AND 10000;
```

Comparison

Feature	B-Tree	B+ Tree
Data pointers	Internal and leaf nodes	Leaf nodes only
Range queries	Good	**Excellent**
Sequential access	Less efficient	**Very efficient**
Tree height	Higher	Usually lower
Large databases	Good	**Better**
Storage efficiency	Moderate	**High**

Conclusion

For large-scale database applications, B+ Tree indexing is generally preferred because it provides efficient search, excellent range queries, and linked leaf nodes for sequential access. This makes it particularly suitable for large databases such as e-commerce, banking, and airline systems.

6. Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Linear Hashing
Linear hashing is a dynamic hashing technique where buckets are split gradually, usually one bucket at a time. It does not require a large directory, so bucket management is simple and gradual.

Extendible Hashing
Extendible hashing uses a directory of pointers to buckets. When a bucket overflows, it is split, and the directory may be doubled. This provides fast access but requires additional directory space.

Bucket Management

Feature	Linear Hashing	Extendible Hashing
Bucket splitting	Gradual, one at a time	Based on bucket overflow
Directory	No directory required	Uses a directory
Overflow handling	Progressive splitting	Bucket splitting + directory expansion
Space overhead	Lower	Higher

```
mysql> CREATE TABLE employees (
    ->     employee_id INT PRIMARY KEY,
    ->     employee_name VARCHAR(100),
    ->     department VARCHAR(50)
    -> );
ERROR 1050 (42S01): Table 'employees' already exists
mysql>
mysql> CREATE INDEX idx_employee_id
    -> ON employees(employee_id);
Query OK, 0 rows affected, 1 warning (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 1

mysql>
mysql> SELECT *
    -> FROM employees
    -> WHERE employee_id = 1050;
```

Conclusion
Linear hashing is simpler and has lower space overhead, while extendible hashing provides faster and more controlled bucket access at the cost of directory storage. For continuously growing data, both are scalable, but extendible hashing is generally better when very fast dynamic access is required.

7. Compare dense indexing and sparse indexing with respect to memory usage and access speed.

Dense Indexing

A dense index contains an index entry for every record in the data file. It requires more memory/storage but provides faster direct access to records.

Sparse Indexing

A **sparse index** contains index entries for only **some records**, typically one entry per data block. It requires less memory but may need an additional search within the data block.

```
mysql> CREATE TABLE students (
    ->     student_id INT PRIMARY KEY,
    ->     student_name VARCHAR(100),
    ->     department VARCHAR(50)
    -> );
ERROR 1050 (42S01): Table 'students' already exists
mysql>
mysql> CREATE INDEX idx_student_name
    -> ON students(student_name);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT *
    -> FROM students
    -> WHERE student_name = 'Arun';
+------------+--------------+------------+
| student_id | student_name | department |
+------------+--------------+------------+
|        103 | Arun         | CSE        |
+------------+--------------+------------+
1 row in set (0.00 sec)

mysql> CREATE TABLE products (
    ->     product_id INT PRIMARY KEY,
    ->     product_name VARCHAR(100),
    ->     price DECIMAL(10,2)
    -> );
ERROR 1050 (42S01): Table 'products' already exists
mysql>
mysql> SELECT *
    -> FROM products
    -> WHERE product_id = 105;
```

Comparison

Feature	Dense Index	Sparse Index
Index entries	Every record	Some records/blocks
Memory usage	**High**	**Low**
Access speed	**Faster**	Slightly slower
Storage requirement	High	Low
Suitable for	Frequently searched data	Large sorted files

Conclusion

Dense indexing provides faster access but consumes more memory, while sparse indexing saves memory and storage but may require additional searching within a block. For very large databases, sparse indexing can reduce index storage significantly.

8. Analyse the advantages and limitations of indexed sequential file organization versus direct file organization.

Indexed Sequential File Organization

Indexed sequential organization stores records in sorted order along with an index. The index provides fast access to records, while the sequential arrangement supports ordered and range-based retrieval.

Advantages:

- Fast direct access using the index.
- Efficient sequential and range retrieval.
- Suitable for large databases.
- Supports both sequential and random access.

Limitations:

- Requires additional storage for indexes.
- Insertions and deletions can be costly.
- Index maintenance adds overhead.

Direct File Organization

Direct file organization uses a hash function to determine the storage location of a record. It provides direct access without searching through other records.

Advantages:

- Very fast direct access.
- Efficient for exact-match searches.
- Insertions and deletions are generally fast.
- No need to maintain sorted records.

Limitations:

- Poor for sequential and range queries.
- Hash collisions can reduce performance.
- Overflow handling may be required.

```
mysql> CREATE TABLE students (
    ->     student_id INT PRIMARY KEY,
    ->     student_name VARCHAR(100),
    ->     department VARCHAR(50)
    -> );
ERROR 1050 (42S01): Table 'students' already exists
mysql>
mysql> CREATE INDEX idx_department
    -> ON students(department);
ERROR 1061 (42000): Duplicate key name 'idx_department'
mysql>
mysql> SELECT *
    -> FROM students
    -> WHERE department = 'CSE';
+------------+--------------+------------+
| student_id | student_name | department |
+------------+--------------+------------+
|        103 | Arun         | CSE        |
|        104 | Rahul        | CSE        |
+------------+--------------+------------+
2 rows in set (0.00 sec)

mysql> SELECT *
    -> FROM students
    -> WHERE student_id = 101;
+------------+--------------+------------+
| student_id | student_name | department |
+------------+--------------+------------+
|        101 | Kavin        | ECE        |
+------------+--------------+------------+
1 row in set (0.00 sec)
```

Comparison

Feature	Indexed Sequential	Direct
Search	Fast	**Very fast**
Range queries	**Excellent**	Poor
Sequential access	**Excellent**	Poor
Insert/Delete	Moderate	**Fast**
Extra storage	Index required	Hash/overflow space
Best use	Ordered and range searches	Exact-match searches

Conclusion

Indexed sequential organization is better when both sequential and random access are required, while direct organization is better when the system mainly performs exact-match searches and requires very fast direct access.

9. Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Single-Level Indexing

Single-level indexing uses one index level to point directly to the data records or blocks. It is simple and suitable for small databases, but searching becomes slower when the index itself becomes very large.

```
mysql> CREATE TABLE students (
    ->     student_id INT PRIMARY KEY,
    ->     student_name VARCHAR(100),
    ->     department VARCHAR(50)
    -> );
ERROR 1050 (42S01): Table 'students' already exists
mysql>
mysql> CREATE INDEX idx_department
    -> ON students(department);
ERROR 1061 (42000): Duplicate key name 'idx_department'
mysql>
mysql> SELECT *
    -> FROM students
    -> WHERE department = 'CSE';
+------------+--------------+------------+
| student_id | student_name | department |
+------------+--------------+------------+
|        103 | Arun         | CSE        |
|        104 | Rahul        | CSE        |
+------------+--------------+------------+
2 rows in set (0.00 sec)
```

Comparison

Feature	Single-Level Indexing	Multi-Level Indexing
Index levels	One	Multiple
Search speed	Fast for small indexes	**Faster for large databases**
Memory requirement	Lower	Higher
Disk I/O	More for large indexes	**Less**
Complexity	Simple	More complex
Suitable for	Small databases	**Large databases**

Conclusion

Single-level indexing is simple and suitable for smaller databases, while multi-level indexing provides more efficient retrieval for large databases by reducing disk I/O and search time.

10. Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Hashing Techniques

Hashing is highly efficient for exact-match queries because a hash function directly maps a search key to a bucket. Average search time is O(1). However, hashing is not suitable for range queries because records are not stored in sorted order.

```
mysql> SELECT *
    -> FROM students
    -> WHERE student_id = 101;
+------------+--------------+------------+
| student_id | student_name | department |
+------------+--------------+------------+
|        101 | Kavin        | ECE        |
+------------+--------------+------------+
1 row in set (0.00 sec)

mysql> CREATE INDEX idx_cgpa
    -> ON students(cgpa);
ERROR 1072 (42000): Key column 'cgpa' doesn't exist in table
mysql>
mysql> SELECT *
    -> FROM students
    -> WHERE cgpa BETWEEN 8.0 AND 9.0;
```

Performance Comparison

Query Type	Hashing	B+ Tree Indexing
Exact-match	**Excellent – O(1) average**	Good – O(log N)
Range query	Poor	**Excellent**
Sequential access	Poor	**Excellent**
Storage overhead	Moderate	Moderate
Best use	Exact searches	Exact + range searches

Cost Analysis

For a large database, hashing can provide faster exact-match retrieval because it directly identifies the bucket. However, for range queries such as price BETWEEN 5000 AND 10000, B+ Tree indexing performs much better because values are maintained in sorted order.

Conclusion

Hashing is preferred for exact-match queries, while B+ Tree indexing is preferred when both exact-match and range queries are required. For general-purpose database systems, B+ Tree indexing provides greater flexibility.